

## Project Assignment #2

### Performance Analysis of LLM Inference Engines

2025/2026

## 1 Introduction

Understanding *why* a system performs the way it does is the core skill of high-performance computing. Collecting metrics is necessary, but not sufficient, i.e., you must be able to **build a performance model**, validate it against measurements, and use it to predict where bottlenecks lie and what optimisations would have the most impact.

In this project, you will choose one of the following distributed inference workloads to run on Deucalion's CPU infrastructure (ARM or x86 nodes; **GPU partitions are not available**):

1. **LLM Inference Engines:** deploy one or more open-source LLM inference engines; benchmark them under varied conditions; and explain the observed performance through hardware-aware analysis (memory bandwidth, compute intensity, threading).
  - **Track A1 — Single-Engine Deep Dive:** pick **one** engine and systematically explore its optimisation space.
  - **Track A2 — Cross-Engine Comparison:** pick **two or more** engines and compare them under identical conditions.

**Important:** In your report you must explicitly state which track (A1 or A2) you chose.

### The Performance-Modelling Requirement

Regardless of your chosen track, your report must include a section where you:

1. **Propose a simple analytical model** that predicts a key metric from hardware and workload parameters. Example:
  - $TPOT \geq \frac{\text{model\_bytes}}{BW_{\text{mem}}}$ , where  $BW_{\text{mem}}$  is the measured DRAM bandwidth on the node, since each decode step must load the full model weights.
2. **Validate it** against your measurements: plot predicted vs. observed values and discuss where the model is accurate and where it breaks.
3. **Use it** to identify the dominant bottleneck and to reason about what optimisation would have the highest impact (even if you cannot implement it).

The model does not need to be perfect. A simple model that captures the right trend and is honest about its limitations is more valuable than a complex model that overfits.

## Deliverables

Your delivery consists of: (i) a single research-style report and (ii) all code, scripts, and configuration files in your GitHub repository.

## 2 LLM Inference Engine Analysis

This track applies the distributed inference concepts from the lectures to a practical setting: serving LLMs on CPU. You will deploy one or more inference engines, benchmark them, and explain the results through hardware-aware analysis.

### 2.1 Models

Because you are running on a CPU, you must use small, quantised models. Suggested models (**optional**):

| Model                      | Parameters | Notes                     |
|----------------------------|------------|---------------------------|
| Qwen2.5-0.5B / 1.5B        | 0.5–1.5B   | Good multilingual support |
| TinyLlama-1.1B             | 1.1B       | Fast, widely benchmarked  |
| SmolLM2-135M / 360M / 1.7B | 0.1–1.7B   | Very small, fast on CPU   |
| Phi-3-mini (3.8B)          | 3.8B       | Larger; INT4 recommended  |
| Gemma-2-2B                 | 2B         | Strong for its size       |

#### Requirements:

- **Minimum number of models:** you must test and benchmark **at least 3 models**.
- **Mandatory baseline model:** one of the three models **must be** Meta-Llama-3.1-8B-Instruct-Q4\_K\_M.gguf.
- **Additional models:** you may choose the other 2+ models from the list above, or from any quantised open-source LLM.

### 2.2 Inference Engines

Choose from the open-source inference engine landscape.<sup>1</sup> Below are engines known to support CPU inference:

| Engine      | Notes                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| llama.cpp   | Excellent CPU support; GGUF format; many quantisation options (Q2–Q8, K-quants). Built-in HTTP server and benchmarking tools. |
| vLLM        | Has a CPU backend. PagedAttention, continuous batching. May require specific build flags for CPU.                             |
| TGI         | Hugging Face's serving stack. CPU support exists but may be limited.                                                          |
| CTranslate2 | Optimised CPU inference with INT8/INT16. Requires model conversion.                                                           |
| MLC LLM     | TVM-based compilation targeting specific CPU architectures.                                                                   |

You may choose other engines not listed here. Document any setup challenges in your report.

**Note that:** getting an engine to compile and run on Deucalion is a legitimate part of the work.

<sup>1</sup>A comprehensive list is maintained at <https://github.com/sihyeong/Awesome-LLM-Inference-Engine>. You are not limited to this list.

## 2.3 Metrics

Collect **at least three** metrics below. The remaining two are optional but will deepen your analysis.

1. **Time to First Token (TTFT):** time from request submission to the first output token. Reflects prefill cost.
2. **Time Per Output Token (TPOT):** average inter-token latency during decode.
3. **Throughput:** total tokens generated per second. Report separately for prefill (prompt tokens/s) and decode (generation tokens/s) where possible.
4. **Peak memory:** peak memory consumed by the serving process during the benchmark. Measure it from SLURM output (MaxRSS), from `/proc/[pid]/status` (VmHWM field), or by sampling with `dstat --mem`.
5. **Goodput:** fraction of requests completed within a latency budget you define (e.g.,  $TTFT < 2\text{ s}$ ,  $TPOT < 200\text{ ms}$ ). Justify the SLA.

## 2.4 Track A1: Single-Engine Deep Dive

Pick **one** inference engine and systematically explore its performance.

### Task 1: Deployment and Baseline

1. Install and configure the engine on a Deucalion node. Document the exact build steps, modules loaded, and any patches required.
2. Serve a model and verify correctness (coherent outputs for test prompts).
3. Run the benchmark workload (Section 3) with default settings. This is your baseline.

### Task 2: Optimisation Exploration

Apply changes **one at a time**, measure, and keep a running comparison table. Explore **at least three** of the following dimensions:

1. **Quantisation levels:** compare formats available in your engine (e.g., FP16, Q8\_0, Q5\_K\_M, Q4\_K\_M, Q2\_K for llama.cpp; INT8, INT4 for others). Report the trade-off between throughput, memory, and output quality.<sup>2</sup>
2. **Threading and parallelism:** vary the number of inference threads (e.g., 1, 4, 8, 16, 32, 48). Identify the point of diminishing returns.
3. **Concurrent requests:** increase simultaneous requests (e.g., 1, 2, 4, 8, 16). Observe how throughput and per-request latency evolve.
4. **Context length:** vary prompt length (128, 512, 1024, 2048 tokens) and observe TTFT and memory impact.
5. **Decode length:** vary maximum generated tokens (64, 128, 256, 512) and observe TPOT stability.
6. **Node architecture:** compare the same configuration on ARM (normal-arm) vs. x86 (normal-x86).

<sup>2</sup>To assess quality degradation you may compute perplexity on a reference corpus (e.g., WikiText-2) or perform a small qualitative evaluation.

**Task 3: Bottleneck Analysis**

- Identify the **primary bottleneck** (memory bandwidth, compute, thread contention, memory capacity). Use profiling evidence.
- Build the performance model from Section 1: predict TPOT from model size, quantisation format, and measured memory bandwidth. Compare the prediction with the measurement.
- Discuss what optimisations the engine **does not implement** but could benefit from (e.g., speculative decoding, KV cache quantisation, NUMA-aware scheduling). Reference the lecture material.

**2.5 Track A2: Cross-Engine Comparison**

Pick **two or more** inference engines and compare them under controlled conditions.

**Task 1: Deployment**

1. Install and configure each engine on the same type of Deucalion node.
2. Load at least **three models** on each engine (same parameter count, same or equivalent quantisation between engines and model runs). Document any format conversions required (e.g., Hugging Face → GGUF, CTranslate2 conversion).
3. Verify correctness: confirm that each engine produces coherent outputs for the same prompts.

**Task 2: Controlled Comparison**

Run the benchmark workload (Section 3) on each engine under identical conditions:

- Same node type, same number of threads, same model, same quantisation level.
- Same prompt set, same generation parameters (temperature, top-p, max tokens).
- Three trials per configuration; report mean  $\pm$  standard deviation.

Vary **at least two** of the following dimensions across engines:

1. Concurrency (1, 4, 8, 16 simultaneous requests).
2. Prompt length (short  $\sim 128$  tokens vs. long  $\sim 1024+$  tokens).
3. Quantisation (matching levels across engines, where supported).

**Task 3: Comparative Analysis**

- For each metric, determine which engine is superior and under what conditions. Use tables and plots.
- Identify the **architectural decisions** that explain the differences (e.g., continuous vs. static batching, PagedAttention vs. contiguous KV cache, threading model, memory allocator).
- Discuss **ease of deployment**: which engine was easier to install, configure, and adapt to the HPC environment? This is a legitimate engineering consideration.
- Build the performance model from Section 1 for at least one engine; discuss whether the other engine's results are consistent with or deviate from the same model.
- Identify missing optimisations in each engine and what impact they would have.

### 3 Benchmarking Protocol

#### A. Standard Benchmark Workload

Prepare a fixed set of prompts for all inference experiments:

- **Short prompts** (~50–128 tokens): simple questions, instructions.
- **Medium prompts** (~256–512 tokens): paragraphs with follow-up questions.
- **Long prompts** (~1024+ tokens): multi-paragraph context with a question.

Include at least 10 prompts per category. Use fixed generation parameters. Store the prompt set in your repository as a JSON file.

**Fixed Comparison Prompts (Mandatory):** All groups **must include and report results for these three standardised prompts**:

1. **Short prompt (baseline):** “What is the capital of France?”
2. **Medium prompt (reasoning):** “Explain the key differences between machine learning and deep learning, and provide an example of each.”
3. **Long prompt (context awareness):** “The transformer architecture, introduced in the paper *Attention is All You Need* by Vaswani et al. in 2017, has become the foundation for most modern large language models. Unlike recurrent neural networks, which process sequences sequentially, transformers process all tokens in parallel using self-attention mechanisms. This allows better parallelisation during training and has proven more effective at capturing long-range dependencies in text. When deploying transformers for inference on CPU, the main bottlenecks are the model’s large memory footprint and the repeated loading of weights during the decoding phase. Based on this context, what are the main challenges when running large transformer models on CPU-only systems, and how might quantisation help address these challenges?”

For each of these three prompts, record: TTFT, TPOT, memory peak, and the generated output.

#### B. Controlled Execution

1. Use the same node type, SLURM allocation, and software modules across all runs within a comparison.
2. Warm up: discard the first run or the first few requests.
3. **Three trials** per configuration; report mean  $\pm$  standard deviation.

#### C. Measurement Infrastructure

- **Track A1 and A2:** write a benchmarking script that sends requests to the engine API (HTTP or CLI), records timestamps for first token and each subsequent token (if streaming), and computes TTFT, TPOT, and throughput. You may use or adapt existing tools (e.g., llama.cpp’s `llama-bench`, vLLM’s `benchmark_serving.py`) but you must understand and document what they measure. Log all raw measurements to CSV/JSON.
- **Resource monitoring:** sample CPU utilisation and memory in a background process during each run (e.g., `dstat --cpu --mem --output run_N.csv`). Optionally collect energy via RAPL.

## D. Possible Tables and Plots

### Track A1:

- Table: TTFT, TPOT, throughput, peak memory per configuration.
- Plot: throughput vs. thread count (or concurrency, quantisation level).
- Plot: TTFT vs. prompt length.
- Plot: memory vs. quantisation level.
- Plot: predicted vs. observed TPOT from your performance model.

### Track A2:

- Table: all metrics side-by-side for each engine per condition.
- Plot: throughput comparison across engines at varying concurrency.
- Plot: latency (TTFT, TPOT) comparison across engines.
- Plot: memory comparison across engines and quantisation levels.

## 4 Deliverables

### Code (GitHub Repository)

#### All subtracks (A1 and A2):

- SLURM job scripts (.sh) for all experiments.
- Benchmarking / measurement scripts.
- A post-processing script or notebook that reads raw measurements and produces the required tables and plots.
- A README.md with clear instructions to reproduce every experiment.

#### Track A additionally:

- Build / installation scripts (or documented instructions) for each engine.
- The prompt dataset (JSON).
- Any model conversion scripts used.

### Report (Research Style)

- **Introduction:** goals, chosen track, and summary of main findings.
- **Background:** relevant techniques from the lectures as applied to CPU-only inference systems (quantisation, batching, KV cache, memory bandwidth, threading). This should demonstrate understanding, not just paraphrase slides.
- **Methodology:** hardware description (node specs, memory, interconnect), software versions, benchmarking protocol, workload design.
- **Results:** tables and plots as specified in the protocol.

- **Performance Model:** the analytical model, its predictions, and validation against measurements (Section 1).
- **Discussion:** bottleneck analysis, which optimisations had the most impact, missing optimisations and their expected effect, limitations of your study.
- **Conclusion:** key findings and practical recommendations.

## 5 Practical Notes for Deucalion

- **Partitions:** use `dev-arm` or `dev-x86` for setup and short tests; `normal-arm` or `normal-x86` for benchmark runs.
- **Compilation (example: llama.cpp):** llama.cpp compiles natively with CMake and GCC. On ARM nodes, enable NEON/SVE optimizations (`-march=native`). On x86 nodes, AVX2/AVX-512 will be auto-detected. For other engines (vLLM, TGI, CTranslate2, MLC LLM), follow their respective build documentation in your repository README.
- **Model download:** download GGUF or Hugging Face models using `wget` or `huggingface-cli` in a short job on a dev node. Quantised models for  $\leq 3B$ -parameter LLMs are typically 1–4 GB.
- **Server and client in one job:** if the engine runs as an HTTP server, launch it in the background within your SLURM job, wait until it is ready (poll the health endpoint), then run the benchmark client, and finally kill the server.
- **Memory:** ARM nodes have 32 GB RAM; x86 nodes have 256 GB RAM.<sup>3</sup> Even unquantised 3B models fit comfortably; the interesting question is how much *less* memory you can use via quantisation without degrading quality or throughput.
- **Time budgets:** a typical inference benchmark (30+ prompts  $\times$  3 trials) completes in under 1 hour for a 1B model on CPU. Plan accordingly.

## 6 Learning Outcomes

You will: (i) deploy and operate distributed inference systems on HPC infrastructure without GPU support; (ii) design and execute a rigorous benchmarking protocol with statistical reporting; (iii) build a simple analytical performance model and validate it against real measurements; (iv) identify bottlenecks through measurement and connect them to the techniques discussed in lectures (quantisation, batching, memory bandwidth, threading); and (v) produce a research-grade report with quantitative evidence and actionable conclusions.

---

<sup>3</sup>Verify these values for the specific partition you use.